

CSGY 9223 Intro to Offensive Security

Week 13 Challenges

nam10102

Challenge - Super Secure Letter

I've strongly encrypted this super secure secret!

See if you can recover the message using the ciphertext letter
nc offsec-chalbroker.osiris.cyber.nyu.edu 1517

Running the binary with a flag.txt file:

```
./super_secure_letter  
Here's my super secure letter! Good luck reading it :)!  
d8af3421e307513cb5ca5144
```

Using binja to see what the binary does.

This looks pretty straight forward. The server reads the flag from the disk, and then encrypts each byte using a pseudo random number generator seeded with the current time squared. A new random number is selected for each character of the flag string.

Each byte is then returned as the hex value of the least significant two bytes. The value is printed out and needs to be decrypted.

The first is to grab each pair of character values returned and convert them to an integer. And with a synchronized random number generator in the solver script, XOR each value to reveal that character of the flag. Continue doing this until the entire flag is revealed.

To synchronize the time values, this was run three or four times. I did add an adjustment parameter, but this was not used when the flag was revealed.

```
Flag: flag{p3rh4p5_n07_50000_53cur3:(_4189ede7292d0f63}
```

Challenge - Pseudo Rand

There's zero chance you can guess my number!

```
nc offsec-chalbroker.osiris.cyber.nyu.edu 1514
```

Running the binary:

```
./pseudo_rand
Can you guess my number?
```

Going to binja...

This one appears to be a pseudo random number generator synchronization problem like the previous challenge. Here, however, the seed is the current time plus 25, and the binary would like the next random number returned.

The binary will take in a string of characters and convert them to an integer do to the comparison. I'll start by copying the last solver script and making my changes.

And the results are:

```
I guess it wasn't that random!
Here's your flag, friend: flag{l00ks_l1k3_th4t_seed_w4snt_gr34t!_3d572178edad2985}
```

Challenge - RSA_2

We encrypted the same message twice, can you decrypt it this time?

```
nc offsec-chalbroker.osiris.cyber.nyu.edu 1516
```

On the surface, this appears to be similar to the recitation exercise we were given. Let's hit the site to see what happens:

RSA 2

Please wait a moment while we generate your ciphertexts ...

```
e1 = 257
n1 = 6623814974974222389545006206646273101861560232672293940161099178649234062957503173916059324642940669126426385
c1 = 9068098541655797531547250908015168417554442041901314202503947022943421864404822869735914035868501467372775885

e2 = 65537
n2 = 6623814974974222389545006206646273101861560232672293940161099178649234062957503173916059324642940669126426385
c2 = 4504795135403422607069936693826770383090462873425809645955957230258292050975440763858885461778650743863976274
```

The site returned e_1, n_1, c_1, e_2 and c_2 . This is the same as the recitation, a common modulus attack. Here there are 2 different public keys using the same n value to generate ciphertext. This would be common if the sender computes two different ciphertexts for two different users using two different public keys, but the same modulus:

- $c_1 = m^{e_1} \pmod{n}$
- $c_2 = m^{e_2} \pmod{n}$

Using the formula: $c_1^a * c_2^b = m^1 = m \pmod{n}$!

We can figure out a and b to decrypt the message m . I copied the recitation script I made and replaced the values with the ones presented by the site and:

```
a = 32641
b = -128
m = b'flag{n1c3_j0b_br34k1ng_T3xtB00k_RSA!_78c78e9d33f0e58a}'
```

Challenge - RSA_1

You don't need a quantum computer to break this one!

```
nc offsec-chalbroker.osiris.cyber.nyu.edu 1515
```

Hitting the site I get:

Please wait a moment while we generate your ciphertext ...

```
e = 5
n = 82207894394255678018829556322353451608034612349299724577941883407964037389029645841377641425979391740446207655
c = 73246603240320346337487822014005910509165152097748001588621908012311718951774432627962584653532661298034189427
```

Can you get the plaintext?

This one appears to be a small exponent problem. This should be pretty easy to compute using gmpy. We will need to find the e th root of c to do this. I constructed a solver with the information provided. To me, e does look pretty small. I guess it all depends on how large m is and if m^e is smaller than n .

I'll use gmpy's `iroot()` function in my solver script:

```
m = gmpy2.iroot(c, e)[0]
```

Running the solver produced:

```
m = b'flag{n0_f4ct0r1ng_r3qu1r3d!_9ca34d9cd3ee7310}'
```

Challenge - Cool Block Challenge

This server is a huge fan of valid padding. See if you can crack the code using its feedback!!

```
nc offsec-chalbroker.osiris.cyber.nyu.edu 1512
```

This one should be fun. I will be a brute force attack using invalid pad to determine the state of each block cipher.

Welcome back friend!

```
IV = 7dc4885cc38ef6fd0181b78f8f48487a
```

```
Ciphertext = 5e726baf0faea53d70bc89543c25c0b6f4f9db36f1cfe531fd07beb7b857dff182f4ff03dcb84fa35acb1b689f3563786aedc
```

This time you have the IV and the Ciphertext. Can you recover the plaintext?

Messages should be hex strings, in the format [IV][Ciphertext].
We're using PKCS#7 padding.
Send me a message!

I start by creating a solver script to read the data. Once this is done, I want to see what happens when I parrot the IV and the Ciphertext back:

```
[DEBUG] Received 0x89 bytes:
b"That's a valid message :)\n"
b'Messages should be hex strings, in the format [IV][Ciphertext].\n'
b'We're using PKCS#7 padding.\n"
b'Send me a message!\n'
```

So the key is to know when a message is valid. This is returned from the site if all the padding checks out. What happens if we alter the last byte?

```
[DEBUG] Received 0x85 bytes:
b'Oh no, bad padding :(\n'
b'Messages should be hex strings, in the format [IV][Ciphertext].\n'
b'We're using PKCS#7 padding.\n"
b'Send me a message!\n'
```

Now I have my two conditions, now it's time to do the Padding Oracle Attack.

DNF

As simple as the process is, I had issues finding all the states for a single block. I tried the first block, the second block and the last block.

For the first block (IV + ciphertext_blocks[0], I could not even get the script to find a single value to produce a valid padding. This made no sense. At least one of the guesses should have produced a 1.

For the last block (entire ciphertext), I could find the first five states, but then I would get invalid padding after all guesses were exhausted. I'm not sure what I was doing wrong, but I spent 3 days and kept spinning my wheels.